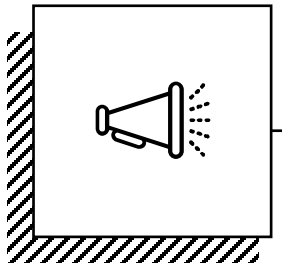
Разрешения

Содержание лекции



1. Основы безопасности: Зачем в Android нужны разрешения?
2. Объявление и запрос: Как задекларировать и запросить разрешение у пользователя.
3. Защита данных: Особенности работы с доступом к камере, микрофону и геолокации.
4. Лучшие практики: Как создавать удобный и безопасный пользовательский опыт.

Основы безопасности: Зачем в Android нужны разрешения?

Основа безопасности: принцип «песочницы» (Sandbox)

- Каждое приложение в Android по умолчанию работает в изоляции.
- У него нет прямого доступа к системе, другим приложениям или данным пользователя.
- Разрешения (Permissions) — это механизм, который позволяет приложению запросить "ключ" для доступа к ресурсам за пределами своей песочницы.

Два типа разрешений

Критерий	Install-time (Обычные)	Runtime (Опасные)
Когда выдаются?	Автоматически при установке	Во время работы приложения, по запросу
Действие пользователя	Неявное согласие (установка)	Явное согласие/отказ в диалоге
Уровень риска	Низкий (не затрагивают личные данные)	Высокий (доступ к камере, контактам, GPS)
Можно ли отозвать?	Нет (только удалив приложение)	Да, в любой момент в настройках
Примеры	INTERNET VIBRATE BLUETOOTH	CAMERA READ_CONTACTS ACCESS_FINE_LOCATION

Два типа разрешений

Группы разрешений:

- Система группирует Runtime-разрешения для удобства пользователя (например, READ_CONTACTS и WRITE_CONTACTS входят в группу CONTACTS).

- Важно: Не следует полагаться на то, что, получив одно разрешение из группы, вы автоматически получите другие. Всегда запрашивайте каждое необходимое разрешение явно.

Объявление и запрос: Как задекларировать и запросить разрешение у пользователя.

Объявление в AndroidManifest.xml

Любое разрешение, которое использует приложение, обязательно должно быть объявлено в манифесте.
Это помогает Play Store и пользователям видеть, к чему ваше приложение может запросить доступ.

```
<manifest ...>
    <uses-permission android:name="android.permission.CAMERA" />
    <uses-permission android:name="android.permission.READ_CONTACTS" />
    <application ...>
        ...
    </application>
</manifest>
```

Разрешение vs. Оборудование (<uses-feature>)

Не путайте запрос разрешения с проверкой наличия оборудования.
<uses-permission>: «Можно ли мне использовать камеру?»
<uses-feature>: «Есть ли на этом устройстве вообще камера?»
Чтобы приложение могли установить на устройства без камеры (например, на ТВ-приставку), укажите, что она не обязательна:

```
<uses-feature android:name="android.hardware.camera"
              android:required="false" />
```

Разрешение vs. Оборудование (<uses-feature>)

В коде всегда проверяйте наличие необязательной функции перед использованием.

```
PackageManager.hasSystemFeature()
```

Главный принцип: запрашивайте разрешение в нужный момент

Не запрашивайте разрешения на старте приложения.
Запрос должен быть контекстным — когда пользователь пытается выполнить действие, требующее этого разрешения.

Пример: Запрашивать доступ к камере только после нажатия на кнопку «Сделать фото».

ПЛОХО: App Start -> Permission Dialog? -> User confused.

ХОРОШО: User clicks "Take Photo" -> Permission Dialog? -> User understands.

Алгоритм запроса Runtime-разрешения

Весь процесс можно описать четырьмя логическими шагами:

1. ПРОВЕРИТЬ: Есть ли у нас уже это разрешение?
2. ОБЪЯСНИТЬ (если нужно): Нужно ли показать пользователю дополнительное объяснение, почему мы просим разрешение? (Это требуется, если он уже один раз отказал).
3. ЗАПРОСИТЬ: Показать системный диалог запроса разрешения.
4. ОБРАБОТАТЬ РЕЗУЛЬТАТ: Выполнить действие, если разрешение получено, или корректно обработать отказ.

Запрос разрешения (современный подход)

Google рекомендует использовать ActivityResultContracts для управления запросами. Это упрощает код и избавляет от «магических чисел» (request codes). Процесс из двух частей:

1. Регистрация callback: В инициализации Activity/Fragment мы создаем «лаунчер» и описываем, что делать с результатом.
2. Запуск запроса: В нужный момент (например, по клику кнопки) вызываем метод launch() у этого лаунчера.

Не забудьте добавить зависимости androidx.activity и androidx.fragment в build.gradle.

Запрос разрешения (современный подход)

```
// 1. Регистрируем обработчик результата
val requestPermissionLauncher =
    registerForActivityResult(ActivityResultContracts.RequestPermission()) {
isGranted: Boolean ->
        if (isGranted) {
            // Разрешение получено. Выполняем действие.
        } else {
            // Пользователь отказал. Объясняем последствия.
        }
    }

// 2. Запускаем запрос по клику на кнопку
button.setOnClickListener {
    requestPermissionLauncher.launch(Manifest.permission.CAMERA)
}
```

Собираем все вместе: полный пример

```
when {
    // 1. ПРОВЕРЯЕМ: разрешение уже есть
    ContextCompat.checkSelfPermission(
        this, Manifest.permission.CAMERA
    ) == PackageManager.PERMISSION_GRANTED -> {
        // Выполняем действие
    }

    // 2. ОБОСНОВАТЬ: показать окно с объяснением, если пользователь уже отказывал
    shouldShowRequestPermissionRationale(Manifest.permission.CAMERA) -> {
        // Показываем свое окно с объяснением, почему нам нужна камера.
        // После этого окна снова вызываем requestPermissionLauncher.launch()
    }

    // 3. ЗАПРАШИВАЕМ: разрешение еще не запрашивалось или стоит "не спрашивать снова"
    else -> {
        requestPermissionLauncher.launch(Manifest.permission.CAMERA)
    }
}
```

Запрос нескольких разрешений сразу

Для запроса нескольких разрешений одновременно используйте контракт ActivityResultContracts.RequestMultiplePermissions().

Результат приходит в виде Map<String, Boolean>, где ключ — имя разрешения, а значение — true (выдано) или false (отклонено).

Запрос нескольких разрешений сразу

```
val multiplePermissionsLauncher = registerForActivityResult(
    ActivityResultContracts.RequestMultiplePermissions()
) { permissions ->
    // Обрабатываем результат для каждого разрешения
    val isCameraGranted = permissions[Manifest.permission.CAMERA] ?: false
    val isLocationGranted =
permissions[Manifest.permission.ACCESS_FINE_LOCATION] ?: false
}

multiplePermissionsLauncher.launch(arrayOf(
    Manifest.permission.CAMERA,
    Manifest.permission.ACCESS_FINE_LOCATION
))
```

Если пользователь отказал: «изящная деградация»

Главное правило: Приложение должно оставаться работоспособным.

Не блокируйте интерфейс: Не показывайте полноэкранное сообщение об ошибке.

Объясняйте последствия: В месте, где функция должна была работать, покажите сообщение. Например, «Для загрузки фото из галереи нужен доступ к файлам».

Уважайте выбор пользователя: Если пользователь дважды нажал «Запретить» (начиная с Android 11), система больше не покажет ему диалог. Это равносильно «Больше не спрашивать». В этом случае shouldShowRequestPermissionRationale вернет false.

Для справки: «ручное» управление запросом (Legacy)

В старом коде вы можете встретить другой подход:

1. Вызов requestPermissions(activity, permissions, requestCode).
2. Переопределение метода onRequestPermissionsResult(...).

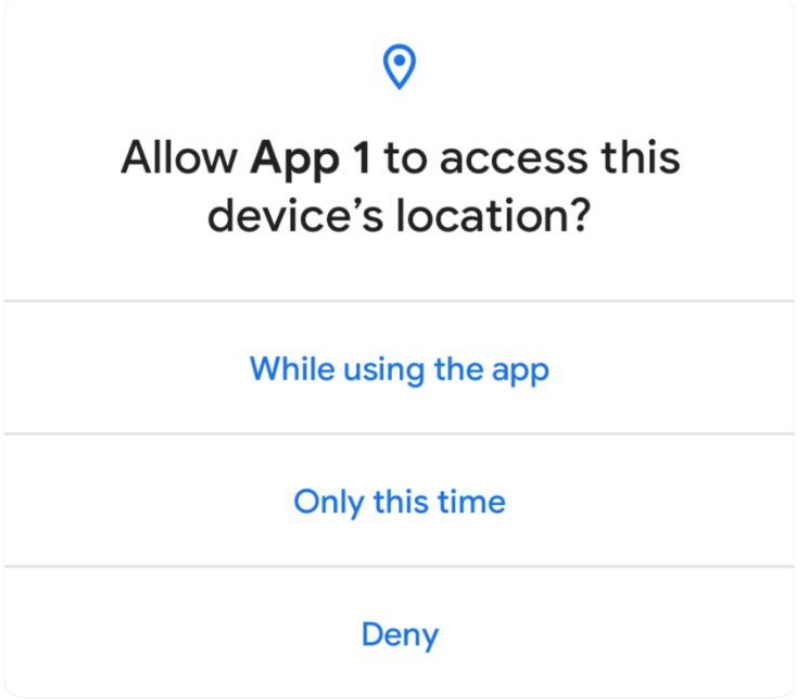
Этот способ считается устаревшим, так как требует ручного управления кодами запросов (requestCode), что усложняет код и может приводить к ошибкам.

Всегда отдавайте предпочтение ActivityResultContracts.

Защита данных: Особенности работы с доступом к камере, микрофону и геолокации.

Функция приватности: одноразовые разрешения (Android 11+)

В диалоге запроса разрешений, связанных с местоположением, камерой и микрофоном, появилась опция «Только в этот раз» (Only this time).



Функция приватности: одноразовые разрешения (Android 11+)

Как это работает:

- Приложение получает временный доступ к данным.
- Доступ сохраняется, пока Activity видна пользователю.
- Если пользователь сворачивает приложение, доступ сохраняется на короткое время.
- Система отзывает разрешение, как только пользователь закрывает приложение или оно долго находится в фоне.

Вывод для разработчика: Не нужно менять логику запроса, но нужно быть готовым, что разрешение «пропадет» при следующем запуске. Проверяйте наличие разрешения каждый раз перед использованием.

Функция приватности: автосброс разрешений (Android 11+)

Если пользователь не взаимодействовал с вашим приложением несколько месяцев, система автоматически отзывает все выданные Runtime-разрешения.

Этот процесс называется гибернацией.

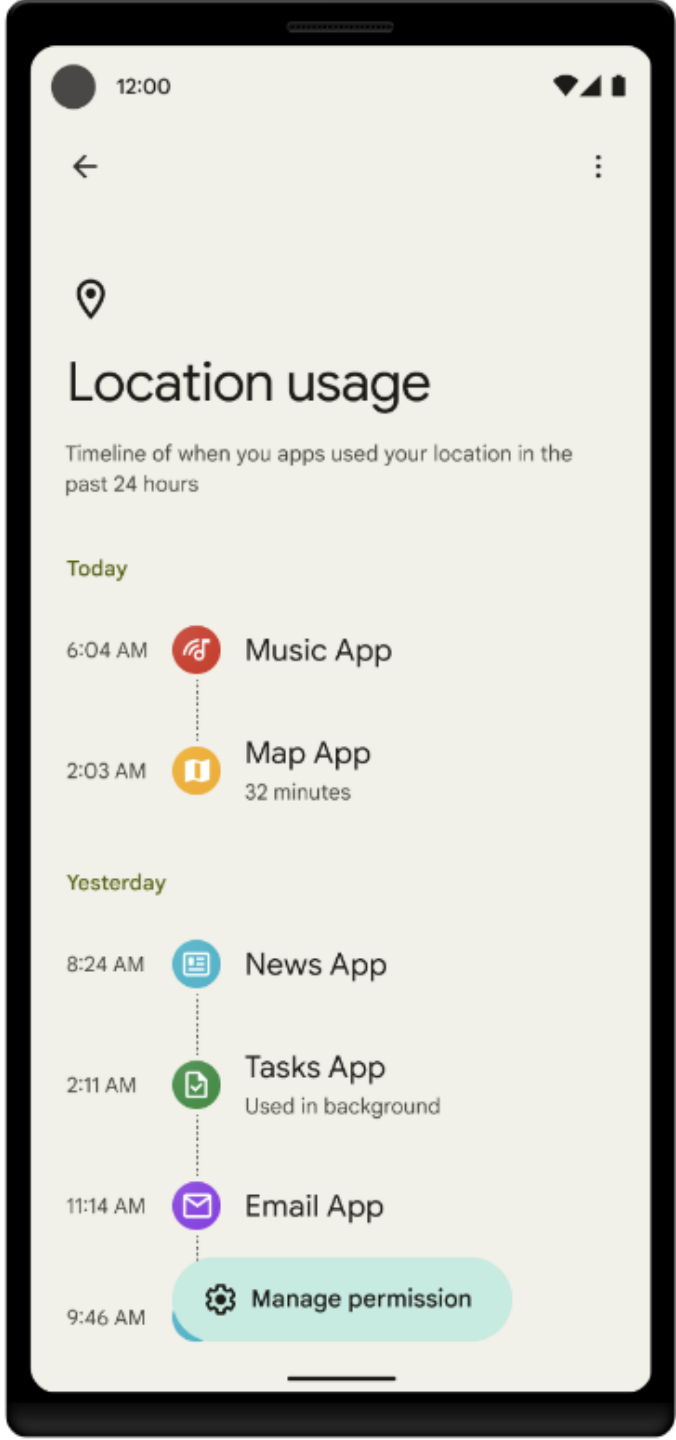
При следующем запуске приложение будет «чистым», как после первой установки.

Вывод для разработчика: Ваше приложение должно корректно обрабатывать ситуацию, когда пользователь возвращается спустя долгое время, и заново проходить через весь флоу запроса разрешений.

Больше контроля для пользователя (Android 12+)

Панель приватности (Privacy Dashboard):

- Экран в настройках, где пользователь видит, какие приложения и когда использовали камеру, микрофон и геолокацию за последние 24 часа.



Больше контроля для пользователя (Android 12+)

Индикаторы в строке состояния:

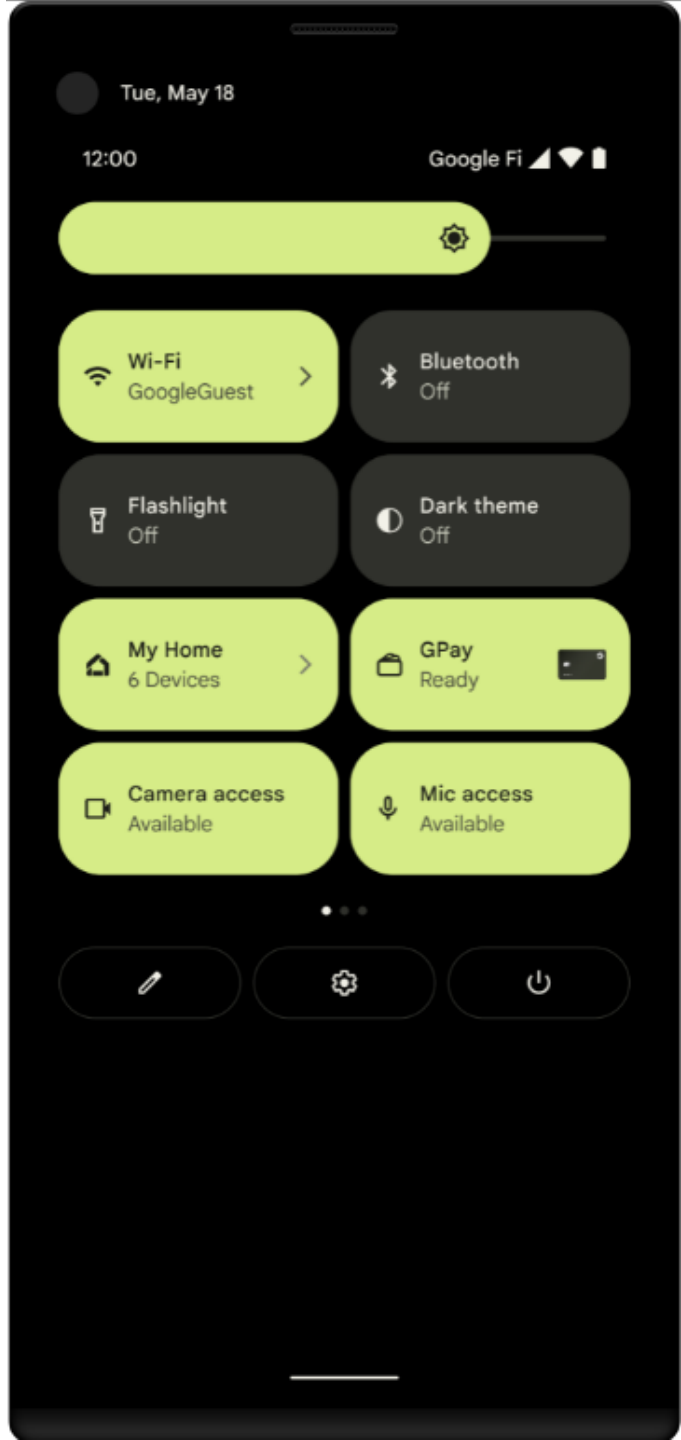
- Зеленая точка в углу экрана сигнализирует, что какое-то приложение прямо сейчас использует камеру или микрофон.



Больше контроля для пользователя (Android 12+)

Глобальные переключатели (Toggles):

- В быстрых настройках пользователь может полностью «выключить» камеру и микрофон для всех приложений на устройстве.



Практика: обработка глобальных переключателей

Если пользователь отключил камеру/микрофон через системный переключатель, ваше приложение (даже имея разрешение!) получит:

- Для камеры: пустой черный кадр.
- Для микрофона: поток с тишиной.

Ваше приложение не должно падать или зависать. Оно должно корректно обработать пустые данные.

Практика: обработка глобальных переключателей

Можно проверить, поддерживает ли устройство такие переключатели:

```
val spm = getSystemService(SensorPrivacyManager::class.java)
val micToggleSupported = spm.supportsSensorToggle(Sensors.MICROPHONE)
val cameraToggleSupported = spm.supportsSensorToggle(Sensors.CAMERA)
```

Особый случай: доступ к геолокации в фоне

Доступ к геолокации, когда приложение не на экране, строго ограничен. Требует отдельного разрешения ACCESS_BACKGROUND_LOCATION.

Правильный порядок запроса:

1. Сначала запросите разрешение на доступ «во время использования» (ACCESS_FINE_LOCATION и/или ACCESS_COARSE_LOCATION).
2. Только после того, как пользователь его предоставит, вы можете (в отдельном запросе) запросить доступ в фоновом режиме.

Запрашивать оба разрешения одновременно запрещено.

Важно: Google Play имеет очень строгие правила. Фоновый доступ разрешен только для приложений, где это является ключевой функцией (например, фитнес-трекеры, навигаторы).

Особые разрешения: системные экраны

Некоторые разрешения (SYSTEM_ALERT_WINDOW, MANAGE_EXTERNAL_STORAGE) считаются настолько мощными, что стандартного запроса для них недостаточно.

Для их получения вы должны перенаправить пользователя на специальный системный экран с помощью Intent. Пользователь должен сам вручную включить опцию.

Особые разрешения: системные экраны

Пример для SYSTEM_ALERT_WINDOW:

```
if (!Settings.canDrawOverlays(this)) {
    val intent = Intent(
        Settings.ACTION_MANAGE_OVERLAY_PERMISSION,
        Uri.parse("package:$packageName")
    )
    startActivity(intent) // Пользователь будет перенаправлен в настройки
}
```

Лучшие практики: Как создавать удобный и безопасный пользовательский опыт.

Лучшая практика №1: Объясняйте, зачем вам разрешение

Исследования показывают: пользователи охотнее дают разрешение, если понимают, для какой конкретной цели оно нужно.

Системный диалог говорит ЧТО вы просите, но не говорит ПОЧЕМУ.

Ваша задача — объяснить «почему»:

- Плохо: Приложению нужен доступ к вашему местоположению.
- Хорошо: Чтобы показать ближайшие рестораны на карте, нам нужен доступ к вашему местоположению.

Используйте окно с обоснованием или подсказки в интерфейсе перед запросом.

Лучшая практика №2: Просите меньше и проверяйте библиотеки

Используйте системные Intent: Вместо того чтобы просить разрешение CAMERA и писать свою реализацию, вы можете запустить системное приложение камеры через Intent. Так вашему приложению разрешение не понадобится вообще.

«Разрешение не требуется»:
Ваше приложение -> Intent -> Приложение Камеры -> Фотография.

Аудит библиотек (SDK):

- Разрешения, которые требуются вашим библиотекам (например, рекламным или аналитическим), пользователь видит как запросы от вашего приложения.
- Проверяйте, какие разрешения добавляют сторонние SDK. Выбирайте те, которые не требуют лишнего.

Лучшая практика №3: Тестируйте все сценарии

Начиная с Android 6.0, вы не можете быть уверены, что у вашего приложения есть все заявленные в манифесте разрешения.

Обязательно протестируйте:

- Сценарий «Согласие»: Пользователь дает разрешение, функция работает.
- Сценарий «Отказ»: Пользователь отказывает, приложение не падает, а «изящно деградирует».
- Сценарий «Отказ навсегда»: Пользователь отказывает дважды. Диалог больше не появляется, приложение предлагает перейти в настройки.
- Сценарий «Разрешение отозвано»: Вы вручную отзываете разрешение в настройках, пока приложение в фоне, и возвращаетесь в него.

Лучшая практика №3: Тестируйте все сценарии

Инструмент: Используйте adb для быстрого управления разрешениями из командной строки^

- Список разрешений и статуса по группам:

```
>>>adb shell pm list permissions -d -g
```

- Предоставить или отозвать одно или несколько разрешений:

```
>>>adb shell pm [grant|revoke] com.your.package PERMISSION
```

Для продвинутых: защита компонентов вашего приложения

Разрешения можно использовать не только для запроса доступа к системным функциям, но и для защиты ваших собственных компонентов от несанкционированного доступа со стороны других приложений.

Вы можете добавить атрибут android:permission="..." к тегам в AndroidManifest.xml:

- <activity>
- <service>
- <provider>
- <receiver>

Если другое приложение попытается запустить ваш защищенный компонент, не имея этого разрешения, система выдаст SecurityException.

[Другое приложение] -X-> [Ваш Service] (доступ закрыт).
[Другое приложение с Permission] ---> [Ваш Service] (доступ открыт).

Подведем итоги

Принцип «песочницы»: Приложения изолированы, разрешения — это ключи к данным за их пределами.

Два типа разрешений: install-time (обычные) и runtime (опасные). Основное внимание — на runtime.

Современный запрос: Используйте ActivityResultContracts. Алгоритм: проверить -> объяснить -> запросить -> обработать.

Приватность пользователя — приоритет: Уважайте выбор («изящная деградация»), будьте готовы к одноразовым разрешениям и автосбросу.

Главное правило: Запрашивайте разрешение в контексте и всегда объясняйте, зачем оно вам нужно.